

This document is a walkthrough of every modular component of the project. The project aims to apply a predictive coding (PC) model that continuously monitors patient data and forecasts the short term future of vital signs is critical. The model's predictions are compared with a baseline RNN (GRU) and ARIMA to determine its relative predictive accuracy. The overview of the project pipeline consists of data preprocessing by creating a windowed and split dataset, implementing the models, training and evaluating the models, and finally an interactive dashboard that displays some results.

Data preprocessing (src/data/data_preprocess.py):

This script converts raw MIMIC-IV demo data into a structured dataset to be used by the models. Load_vitals loads chartevents and joins it with d_items. d_items is a table that maps numeric itemids to readable labels as many of the chartevents are stored as ID numbers. The vitals are then extracted and then pivoted onto a wide format, so one time step contains each vital. The dataset is then placed into 1 hour intervals and forward filling is used for any missing values. This prevents obtaining future information in the time interval. To prevent leakage, the dataset is split by patient ID at 70/15/15 train, validation, and test sets. The training data is finally normalized. A scaler.pkl is saved to keep the mean and standard deviation of each vital sign to un-normalize later in the project. The split datasets are saved in train.csv, val.csv, and test.csv.

Windowed dataset (src/data/window_dataset.py)

Creates a class that converts the processed data into a fixed length sample the models will use to learn. It first sorts by patient and time so all patients are grouped together and each group is chronologically in order. It then groups by the specific hospital admission ID and specific stay ID as patients may have been in the ICU multiple times during one hospital stay, and multiple hospital stays. Each stay obtains a sliding window of specified length as the input and sets a specified future prediction horizon as the target.

PC GRU model (src/models/pc_gru.py)

Implements the predictive coding GRU model. At each time step, the model generates a prediction of the current input and computes a prediction error (difference between predicted and true input). This error signal is used in the hidden state update.

RNN GRU baseline (src/models/gru_baseline.py)

Implements a single layer GRU that processes the input window of vital signs and produces a hidden layer each time step. In the forward pass, the model takes the output and passes it through a dropout layer for regularization, then a linear layer to predict the future vitals. The GRU uses backpropagation to learn.

ARIMA baseline (src/models/arima_baseline.py)

Implements ARIMA, a classical time series model that predicts each vital independently using a univariate model.

Train (src/training/train.py)

Contains the main training and evaluation loops for the PC and baseline GRU models.

Train PC model (src/training/train_pc_gru.py)

Implements training pipeline for PC model across all window sizes and prediction horizons using the dataset splits. Trains using train.py and calculates metrics using metrics.py. It also generates scatterplots of predictions against targets.

Train GRU model (src/training/train_gru.py)

Implements training pipeline for the GRU model. It similarly trains using train.py, calculates metrics using metrics.py, and generates scatterplots of predictions against targets.

Train windowed ARIMA model (src/training/train_arima_windowed.py)

Implements the training pipeline for the windowed ARIMA by constructing fixed length windows to make a fair comparison with previous models.

Dashboard (src/dashboard/dashboard.py)

Small interactive Streamlit dashboard that loads processed patient splits, model checkpoints, and metrics. It also fits ARIMA baseline live on the selected window. It lets the user choose a split, ICU stay, window size, prediction horizon, vital sign, and models to compare.

Metrics (src/evaluation/metrics.py)

Calculates the metrics used for model comparison. The metrics used are MAE, RMSE, Pearson correlation coefficient, and per vital MAE. Direction accuracy is also calculated to determine if the model is able to correctly predict direction of change in a vital's trend rather than specific value.

Results, tables, plots (src/evaluation/results_tables_plots.py)

Combines all model outputs and selects the best performing epoch for each model based on validation loss. It generates summary tables, determines the best model per metric. It also creates plots of train/val loss curves.